Lab DA-6: File-System Internals

Inodes and File Allocation in xv6 (VTOP Lab DA-9, 10)

Name: Mishank Shah
Registration No.: 24BCE2467 (individual submission)
Platform: xv6-public (x86) on QEMU, built and run under WSL Ubuntu
Source: xv6/ | Test program: xv6/fstest.c | New system call: bmapinfo()

Contents

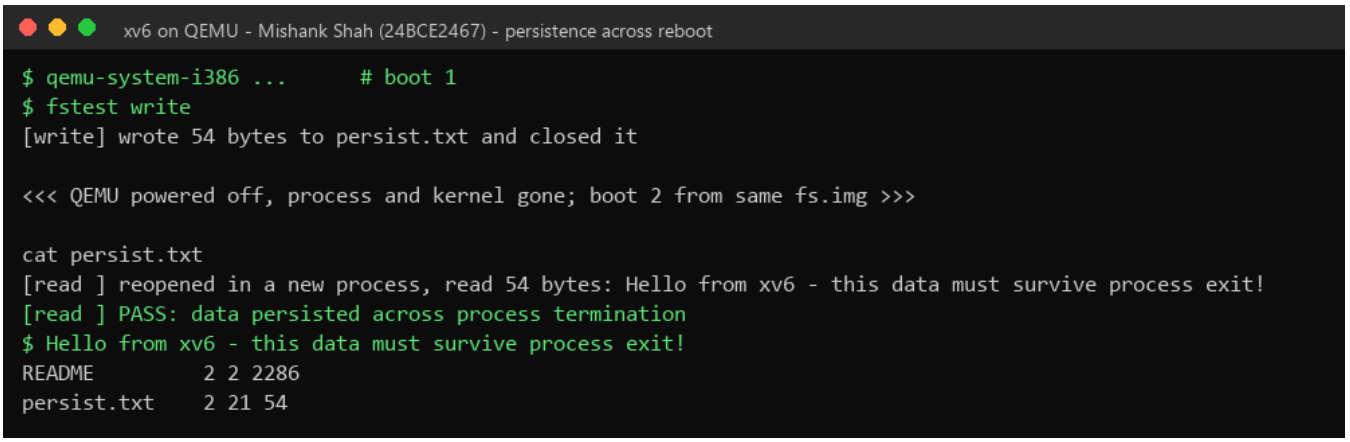
1. File creation and persistence 2. stat and inode fields 3. fd, struct file, inode, disk blocks 4. bmap(): direct and indirect mapping 5. Multi-block write and verification 6. GDB session 7. Code listings 8. Video walkthrough

Overview of what was added to xv6

fstest.c (new user program) exercises every task. **bmapinfo(fd, &info)** (new system call, syscall number 23, in sysfile.c) returns the open file's inode block map: its size, the 12 direct pointers plus the indirect pointer (`ip->addrs[]`), and the 128 entries stored inside the indirect block. It lets user space see exactly which disk blocks hold a file. Files changed: sysfile.c, syscall.c, syscall.h, usys.S, user.h, Makefile; new: bmapinfo.h, fstest.c.

Task 1. Create, write, close, reopen: persistence

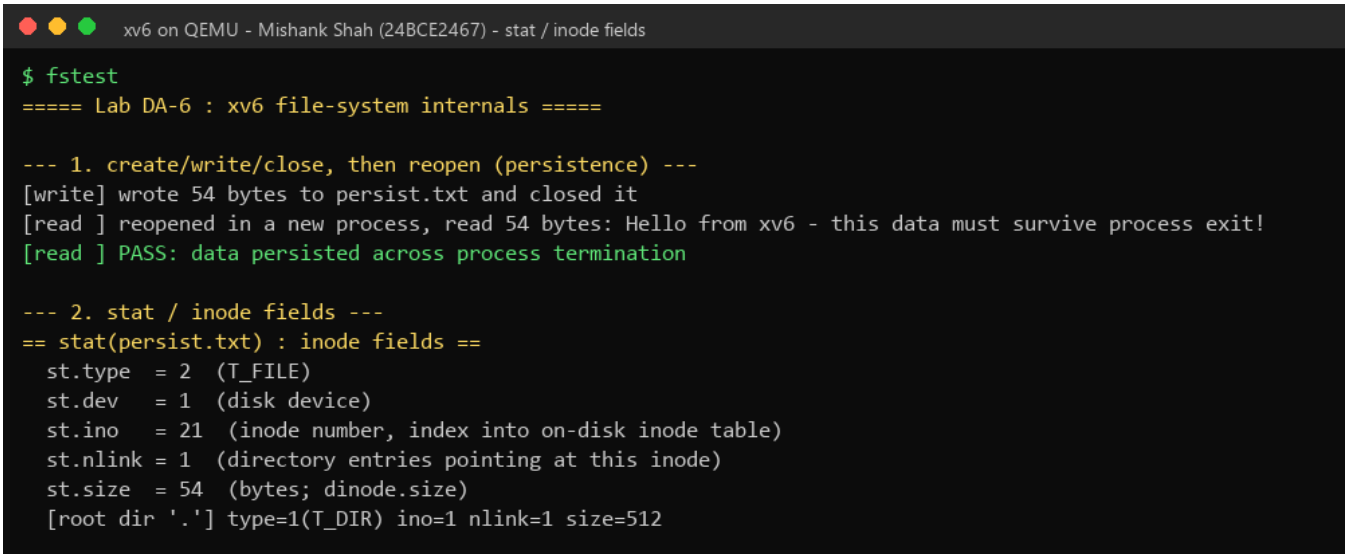
fstest 'write' creates persist.txt, writes a 54-byte message and closes it. QEMU is then shut down completely (process and kernel gone). A fresh QEMU boot (same fs.img) runs 'fstest read', which reopens the file and compares the content; data survives because it was committed through the log (log.c) to the disk image.



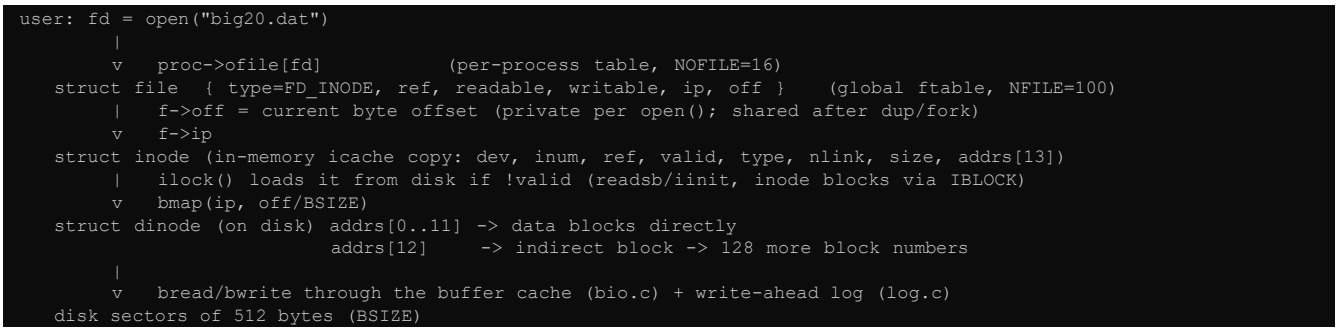
Task 2. stat and inode fields

fstat() fills struct stat from the inode (stati() in fs.c). Mapping to the on-disk `struct dinode` (fs.h):

stat field	inode field	Meaning
type	dinode.type	T_DIR=1, T_FILE=2, T_DEV=3
dev	ip->dev	disk device the inode lives on
ino	ip->inum	index into the on-disk inode table (inode blocks start after the log)
nlink	dinode.nlink	directory entries referring to the inode (file freed when 0 and no open refs)
size	dinode.size	file length in bytes
(hidden)	dinode.addrs[13]	12 direct block numbers + 1 indirect block number



Task 3. File descriptor to struct file to inode to disk blocks



Call path for write(): sys_write (sysfile.c) → filewrite (file.c): splits the request so one log transaction never exceeds MAXOPBLOCKS → writei (fs.c) → for each block calls **bmap()** to translate (offset/BSIZE) to a disk block → bread, memmove, log_write. **Reading** is the mirror image through fileread → readi. The same ip is shared by every struct file that opens the same path, which is why the in-memory inode has a reference count and a sleeplock.

The new bmapinfo() syscall makes this chain visible: it takes an fd, follows `proc->ofile[fd]` → `struct file` → `ip` and returns ip->addrs.

```
xv6 on QEMU - Mishank Shah (24BCE2467) - fd -> struct file -> inode -> blocks (bmapinfo)
$ fstest
== block map of big20.dat (fd=3 -> struct file -> inode 22) size=10240 ==
  addrs[0] (direct)   -> block 751
  addrs[1] (direct)   -> block 752
  addrs[2] (direct)   -> block 753
  addrs[3] (direct)   -> block 754
  addrs[4] (direct)   -> block 755
  addrs[5] (direct)   -> block 756
  addrs[6] (direct)   -> block 757
  addrs[7] (direct)   -> block 758
  addrs[8] (direct)   -> block 759
  addrs[9] (direct)   -> block 760
  addrs[10] (direct)   -> block 761
  addrs[11] (direct)   -> block 762
  addrs[12] (INDIRECT) -> block 763, which holds:
      indirect[0] -> block 764
      indirect[1] -> block 765
      indirect[2] -> block 766
      ...
      indirect[7] -> block 771
  => 12 direct + 8 indirect data blocks (+1 indirect-pointer block)
```

Task 4. bmap() in fs.c: direct and indirect mapping

```
// The content (data) associated with each inode is stored
// in blocks on the disk. The first NDIRECT block numbers
// are listed in ip->addrs[].  The next NINDIRECT blocks are
// listed in block ip->addrs[NDIRECT].

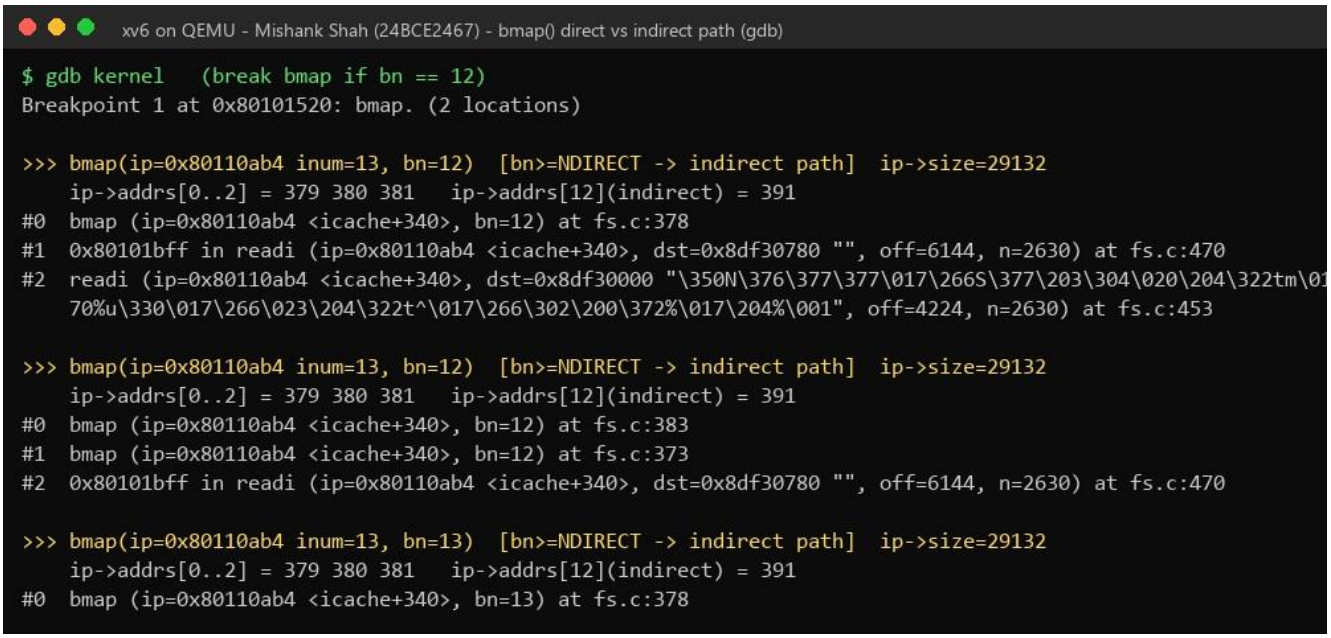
// Return the disk block address of the nth block in inode ip.
// If there is no such block, bmap allocates one.
static uint
bmap(struct inode *ip, uint bn)
{
  uint addr, *a;
  struct buf *bp;

  if(bn < NDIRECT){
    if((addr = ip->addrs[bn]) == 0)
      ip->addrs[bn] = addr = balloc(ip->dev);
    return addr;
  }
  bn -= NDIRECT;

  if(bn < NINDIRECT){
    // Load indirect block, allocating if necessary.
    if((addr = ip->addrs[NDIRECT]) == 0)
      ip->addrs[NDIRECT] = addr = balloc(ip->dev);
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;
    if((addr = a[bn]) == 0){
      a[bn] = addr = balloc(ip->dev);
      log_write(bp);
    }
    brelse(bp);
    return addr;
  }

  panic("bmap: out of range");
}
```

Direct: if bn < 12, the block number is stored in ip->addrs[bn] itself; if 0, balloc() allocates a free block (bitmap scan) and records it. **Indirect:** for 12 ≤ bn < 140, bn is reduced by 12, addrs[12] points to one extra block holding 128 4-byte block numbers (BSIZE/sizeof(uint)); the indirect block is allocated on first use, then a[bn] is looked up (or allocated and the change logged with log_write). **Limits:** MAXFILE = 12 + 128 = 140 blocks = 71,680 bytes; beyond that bmap panics with 'out of range'. Small files cost no extra metadata block; a file larger than 6 KB (12 × 512) pays one extra block.



Why allocation is lazy

bmap() allocates only when a block is first touched, so writes extend the file block-by-block and sparse reads of unallocated regions see zeros. In the output below, big20.dat (20 blocks) uses 12 direct blocks + 1 indirect pointer block + 8 indirect-mapped data blocks; the blocks are contiguous (751..771) because the file system was fresh and balloc takes the first free bit.

Task 5. Writing across multiple block boundaries and verifying

fstest part 3 writes N blocks of 512 bytes; block b is filled with the pattern (b*7+i) mod 251, so every block differs. The file is closed, reopened and read back, and each byte is compared. Tests:

(a) 20 blocks: crosses the direct/indirect boundary at block 12. (b) 140 blocks = MAXFILE: the largest file xv6 can store, fills every pointer. (c) an unaligned 100-byte write at offset 462 which straddles the boundary between block 0 and block 1 (the partial-block path in writei: m = min(n-tot, BSIZE - off%BSIZE)).

xv6 on QEMU - Mishank Shah (24BCE2467) - 20 blocks (direct + indirect)

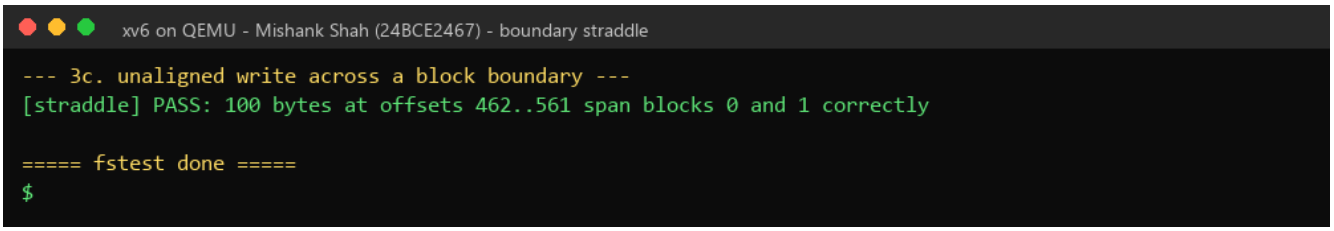
```
--- 3. multi-block write/read-back ---
  wrote logical block 0  (direct)
  wrote logical block 11  (direct)
  wrote logical block 12  (INDIRECT)
  wrote logical block 19  (INDIRECT)
[multi] wrote 20 blocks = 10240 bytes (crosses the direct->indirect boundary at block 12)
[multi] PASS: all 20 blocks read back identical

== block map of big20.dat (fd=3 -> struct file -> inode 22) size=10240 ==
  addrs[0] (direct)   -> block 751
  addrs[1] (direct)   -> block 752
  addrs[2] (direct)   -> block 753
  addrs[3] (direct)   -> block 754
  addrs[4] (direct)   -> block 755
  addrs[5] (direct)   -> block 756
  addrs[6] (direct)   -> block 757
  addrs[7] (direct)   -> block 758
  addrs[8] (direct)   -> block 759
  addrs[9] (direct)   -> block 760
  addrs[10] (direct)   -> block 761
  addrs[11] (direct)   -> block 762
  addrs[12] (INDIRECT) -> block 763, which holds:
      indirect[0] -> block 764
      indirect[1] -> block 765
      indirect[2] -> block 766
      ...
      indirect[7] -> block 771
  => 12 direct + 8 indirect data blocks (+1 indirect-pointer block)
```

xv6 on QEMU - Mishank Shah (24BCE2467) - MAXFILE=140 blocks

```
--- 3b. largest possible file (MAXFILE = 140 blocks) ---
  wrote logical block 0  (direct)
  wrote logical block 11  (direct)
  wrote logical block 12  (INDIRECT)
  wrote logical block 139  (INDIRECT)
[multi] wrote 140 blocks = 71680 bytes (crosses the direct->indirect boundary at block 12)
[multi] PASS: all 140 blocks read back identical

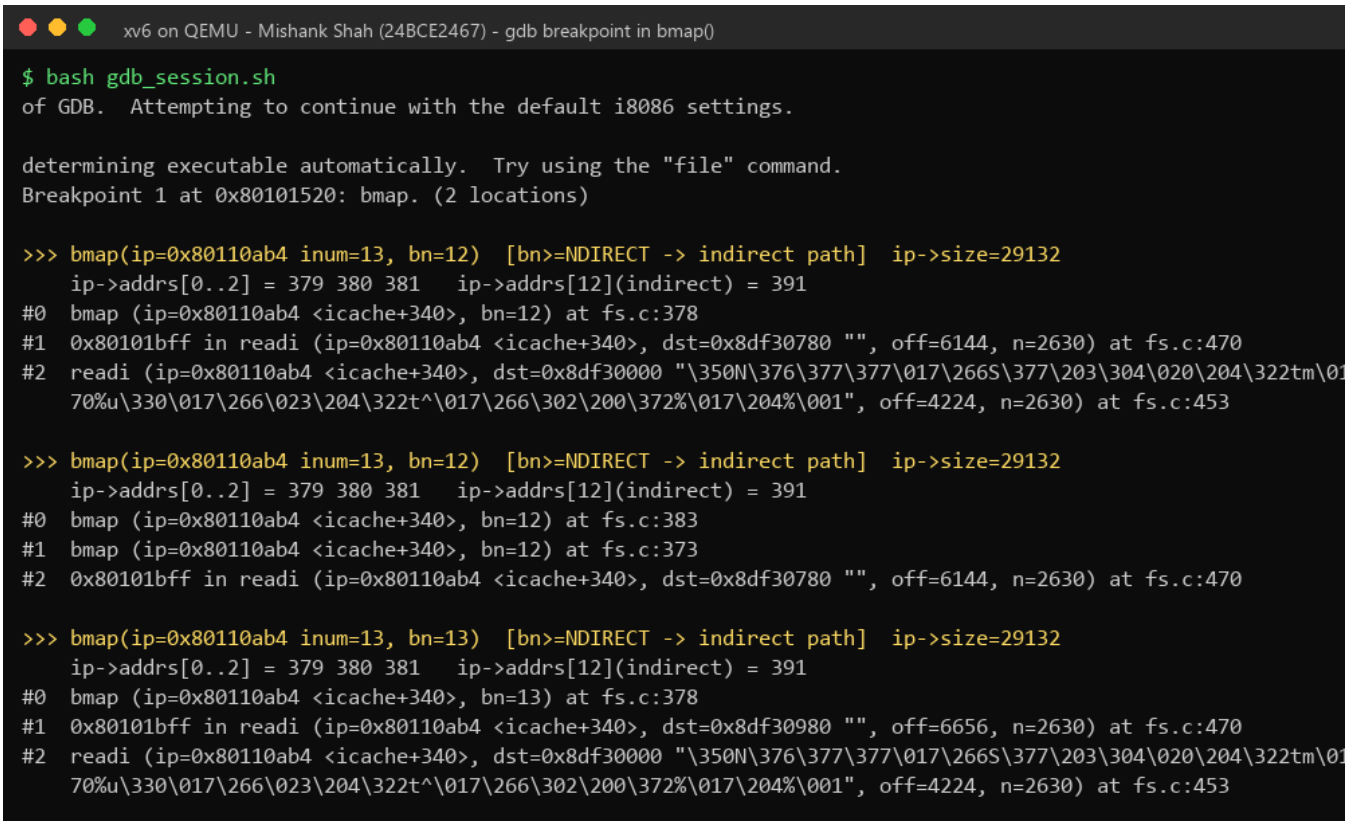
== block map of bigmax.dat (fd=3 -> struct file -> inode 23) size=71680 ==
  addrs[0] (direct)   -> block 772
  addrs[1] (direct)   -> block 773
  addrs[2] (direct)   -> block 774
  addrs[3] (direct)   -> block 775
  addrs[4] (direct)   -> block 776
  addrs[5] (direct)   -> block 777
  addrs[6] (direct)   -> block 778
  addrs[7] (direct)   -> block 779
  addrs[8] (direct)   -> block 780
  addrs[9] (direct)   -> block 781
  addrs[10] (direct)   -> block 782
  addrs[11] (direct)   -> block 783
  addrs[12] (INDIRECT) -> block 784, which holds:
      indirect[0] -> block 785
      indirect[1] -> block 786
      indirect[2] -> block 787
      ...
      indirect[127] -> block 912
  => 12 direct + 128 indirect data blocks (+1 indirect-pointer block)
```



Result: all comparisons PASS; the bmapinfo dump confirms the mapping 12 direct + 128 indirect for the maximum file.

Task 6. GDB session: breakpoint in bmap()

QEMU started with -S -gdb tcp::26111; gdb loads the kernel symbols and sets `break bmap if bn == 12 || bn == 13 || bn == 139`, i.e. only the indirect path. It fires when exec/readi load a binary larger than 6 KB (here inode 13 and 19 are user programs: their ip->addrs[12] already holds an indirect block) and shows the call chain bmap ← readi ← ... and line numbers 378 (direct test) / 383 (bn -= NDIRECT) in fs.c.



Reproduce: `bash gdb_session.sh` (or `make qemu-gdb` in one terminal and `gdb kernel` + `target remote :26000` in another).

Key observations / viva answers

• Why does the file persist? Data goes bwrite -> log -> in-place disk blocks inside fs.img; closing the file only drops references, exiting the process closes all fds, and nothing is erased.

• Why are inode numbers small? Inodes live in a fixed table after the superblock/log; ialloc takes the first free (type==0) slot, ino 1 is the root directory.

• What is the max file size and why? 140 blocks (71,680 B): 12 direct + one indirect block of 128 pointers. A double-indirect block (as in real Unix) would be needed to go bigger.

• What does nlink do? In creates another directory entry for the same inode and nlink++; unlink decrements it; the inode is truncated (itrunc frees blocks) only when nlink==0 and ref==0.

• Why is each write limited in chunks? The log can hold only LOGSIZE = 30 blocks, so filewrite writes at most ((MAXOPBLOCKS-1-1-2)/2)*BSIZE per transaction.

Code listing: fstest.c

```
// Lab DA-6: file-system internals (inodes, file allocation, bmap).
//   fstest          -> runs all parts
//   fstest write    -> part 1 only (create + write file "persist.txt")
//   fstest read     -> part 1b (reopen in a NEW process and verify persistence)
#include "types.h"
#include "stat.h"
#include "user.h"
#include "fcntl.h"
#include "fs.h"
#include "bmapinfo.h"

#define MSG "Hello from xv6 - this data must survive process exit!\n"

static void
fail(char *m)
{
  printf(1, "FAIL: %s\n", m);
  exit();
}

void
part_write(void)
{
  int fd = open("persist.txt", O_CREATE | O_WRONLY);
  if(fd < 0) fail("create persist.txt");
  if(write(fd, MSG, strlen(MSG)) != strlen(MSG)) fail("write");
  close(fd);
  printf(1, "[write] wrote %d bytes to persist.txt and closed it\n", strlen(MSG));
}

void
part_read(void)
{
  char buf[100];
  int n, fd = open("persist.txt", O_RDONLY);
  if(fd < 0) fail("reopen persist.txt");
  n = read(fd, buf, sizeof(buf) - 1);
  buf[n] = 0;
  close(fd);
  printf(1, "[read ] reopened in a new process, read %d bytes: %s", n, buf);
  if(n == strlen(MSG) && strcmp(buf, MSG) == 0)
    printf(1, "[read ] PASS: data persisted across process termination\n");
  else
    fail("persisted data mismatch");
}

void
part_stat(void)
{
  struct stat st;
  int fd = open("persist.txt", O_RDONLY);
  if(fd < 0 || fstat(fd, &st) < 0) fail("fstat");
  printf(1, "\n== stat(persist.txt) : inode fields ==\n");
  printf(1, "  st.type  = %d  (%s)\n", st.type, st.type == T_FILE ? "T_FILE" : st.type == T_DIR ? "T_DIR" : "T_DEV");
  printf(1, "  st.dev   = %d  (disk device)\n", st.dev);
  printf(1, "  st.ino   = %d  (inode number, index into on-disk inode table)\n", st.ino);
  printf(1, "  st.nlink = %d  (directory entries pointing at this inode)\n", st.nlink);
  printf(1, "  st.size  = %d  (bytes; dinode.size)\n", st.size);
  close(fd);

  fd = open(".", O_RDONLY);
  fstat(fd, &st);
  printf(1, "  [root dir '.'] type=%d(T_DIR) ino=%d nlink=%d size=%d\n", st.type, st.ino, st.nlink, st.size);
  close(fd);
}

// Show fd -> struct file -> inode -> data blocks using bmapinfo().
void
show_map(int fd, char *name)
{
  static struct bmapinfo bi;
  struct stat st;
```

```
  int i, nd = 0, ni = 0;

  if(bmapinfo(fd, &bi) < 0) fail("bmapinfo");
  fstat(fd, &st);
  printf(1, "\n== block map of %s (fd=%d -> struct file -> inode %d) size=%d ==\n", name, fd, st.ino, bi.size);
  for(i = 0; i < NDIRECT; i++)
    if(bi.addrs[i]){
      printf(1, "  addrs[%d] (direct)     -> block %d\n", i, bi.addrs[i]);
      nd++;
    }
  if(bi.addrs[NDIRECT]){
    printf(1, "  addrs[%d] (INDIRECT) -> block %d, which holds:\n", NDIRECT, bi.addrs[NDIRECT]);
    for(i = 0; i < NINDIRECT; i++)
      if(bi.indirect[i]){
        if(ni < 3 || bi.indirect[i+1] == 0)
          printf(1, "       indirect[%d] -> block %d\n", i, bi.indirect[i]);
        else if(ni == 3)
          printf(1, "       ...\n");
        ni++;
      }
  }
  printf(1, "  => %d direct + %d indirect data blocks (+%d indirect-pointer block)\n",
         nd, ni, bi.addrs[NDIRECT] ? 1 : 0);
}

// Write NBLK blocks, each filled with a block-specific pattern, then read back and verify.
void
part_multiblock(int nblk, char *name)
{
  char buf[BSIZE];
  int fd, b, i;

  fd = open(name, O_CREATE | O_RDWR);
  if(fd < 0) fail("create big file");
  for(b = 0; b < nblk; b++){
    for(i = 0; i < BSIZE; i++)
      buf[i] = (char)((b * 7 + i) % 251);
    if(write(fd, buf, BSIZE) != BSIZE) fail("write block");
    if(b == 0 || b == NDIRECT - 1 || b == NDIRECT || b == nblk - 1)
      printf(1, "  wrote logical block %d  (%s)\n", b,
             b < NDIRECT ? "direct" : "INDIRECT");
  }
  printf(1, "[multi] wrote %d blocks = %d bytes (crosses the direct->indirect boundary at block %d)\n",
         nblk, nblk * BSIZE, NDIRECT);
  close(fd);

  fd = open(name, O_RDONLY);
  if(fd < 0) fail("reopen big file");
  for(b = 0; b < nblk; b++){
    if(read(fd, buf, BSIZE) != BSIZE) fail("short read");
    for(i = 0; i < BSIZE; i++)
      if(buf[i] != (char)((b * 7 + i) % 251)){
        printf(1, "FAIL: mismatch in block %d byte %d\n", b, i);
        exit();
      }
  }
  printf(1, "[multi] PASS: all %d blocks read back identical\n", nblk);
  show_map(fd, name);
  close(fd);
}

// Unaligned write that straddles a block boundary.
void
part_boundary(void)
{
  char out[100], in[100];
  int i, fd = open("straddle.txt", O_CREATE | O_RDWR);
  for(i = 0; i < 100; i++) out[i] = 'A' + (i % 26);
  // pad to 462 bytes, so the next 100 bytes cross the 512-byte boundary
  for(i = 0; i < 462; i += 1) write(fd, "x", 1);
  write(fd, out, 100);
  close(fd);
```

```
  fd = open("straddle.txt", O_RDONLY);
  for(i = 0; i < 462; i++) read(fd, in, 1);
  read(fd, in, 100);
  close(fd);
  for(i = 0; i < 100; i++)
    if(in[i] != out[i]) fail("boundary-straddling write corrupted");
  printf(1, "[straddle] PASS: 100 bytes at offsets 462..561 span blocks 0 and 1 correctly\n");
}

int
main(int argc, char *argv[])
{
  if(argc > 1 && strcmp(argv[1], "write") == 0){ part_write(); exit(); }
  if(argc > 1 && strcmp(argv[1], "read") == 0){ part_read(); exit(); }

  printf(1, "===== Lab DA-6 : xv6 file-system internals =====\n");
  printf(1, "\n--- 1. create/write/close, then reopen (persistence) ---\n");
  part_write();
  part_read();
  printf(1, "\n--- 2. stat / inode fields ---");
  part_stat();
  printf(1, "\n--- 3. multi-block write/read-back ---\n");
  part_multiblock(20, "big20.dat");      // 12 direct + 8 indirect
  printf(1, "\n--- 3b. largest possible file (MAXFILE = %d blocks) ---\n", MAXFILE);
  part_multiblock(MAXFILE, "bigmax.dat");
  printf(1, "\n--- 3c. unaligned write across a block boundary ---\n");
  part_boundary();
  printf(1, "\n===== fstest done =====\n");
  exit();
}
```

Code listing: bmapinfo.h and sys_bmapinfo (sysfile.c)

```
// Result of the bmapinfo() system call: a snapshot of an inode's block map.
struct bmapinfo {
  uint size;                // file size in bytes
  uint addrs[NDIRECT+1];    // ip->addrs: 12 direct blocks + 1 indirect block pointer
  uint indirect[NINDIRECT]; // contents of the indirect block (0 if none)
};

// bmapinfo(fd, &info): copy the file's inode block map (direct + indirect) to user space.
#include "buf.h"
#include "bmapinfo.h"
int
sys_bmapinfo(void)
{
  struct file *f;
  struct bmapinfo *bi;
  struct buf *b;
  int i;
  if(argfd(0, 0, &f) < 0 || argptr(1, (void*)&bi, sizeof(*bi)) < 0)
    return -1;
  if(f->type != FD_INODE)
    return -1;
  ilock(f->ip);
  bi->size = f->ip->size;
  for(i = 0; i <= NDIRECT; i++)
    bi->addrs[i] = f->ip->addrs[i];
  for(i = 0; i < NINDIRECT; i++)
    bi->indirect[i] = 0;
  if(f->ip->addrs[NDIRECT]){
    b = bread(f->ip->dev, f->ip->addrs[NDIRECT]);
    for(i = 0; i < NINDIRECT; i++)
      bi->indirect[i] = ((uint*)b->data)[i];
    brelse(b);
  }
  iunlock(f->ip);
  return 0;
}
```

Registration of the syscall

```
syscall.h : #define SYS_bmapinfo 23
syscall.c : extern int sys_bmapinfo(void);    [SYS_bmapinfo] sys_bmapinfo,
usys.S    : SYSCALL(bmapinfo)
user.h    : int bmapinfo(int, struct bmapinfo*);
Makefile  :  fstest added to UPROGS
```

Video walkthrough

Team member	Timestamps
Mishank Shah (24BCE2467)	00:00 - end

Video link:
https://drive.google.com/file/d/1DxuaQW_IVmo7nF8pmxxcAwrZtVa8TXVq/view?usp=drive_link

Github Link: